



PYTHON PROGRAMMING

(355)

NATIONAL 2025

APPLICATION KNOWLEDGE:

TOTAL POINTS

_____ (480 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Contestant must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

Dungeon Master's Forge: Chronicles of Salvation

The saga continues, inviting you into the realm of advanced game development. This iteration bestows upon you the power to not only guide adventurers through the perilous depths but also to offer them a lifeline - the ability to save their journey. With this newfound capability, your task expands to include crafting a mechanism that allows players to pause their quest at will and resume their valorous exploits later, adding a layer of strategy and persistence to the dungeon crawl experience.

Contest Requirements:

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

The provided student file, *NationalPython.py*, is the file in which you will write your solution. The student file *GameSaves.txt* is the file that your solution will read from and write to.

1. Create a folder using your *contestant number* as the name of the folder.
2. Copy the following files into this folder: *NationalPython.py* and *GameSaves.txt*. Both files *must* be present.
3. If submitting your test with a flash drive, copy your folder to the flash drive.
4. If submitting your test online, zip your folder and upload the zipped folder to the test submission website.
5. You will need to use a local Python IDE to complete this exam. No online interpreters for Python are allowed.
6. The graders will *not* compile or alter your source code if it does not run as-is.
7. Submissions that do *not* contain source code and/or files other than Python files will *not* be graded.

What you will be doing:

1. **Include your contestant number as a comment at the top of the main source code file.**
2. Implement new ***save_game*** function into program.
3. Convert list of characters in function ***load_characters*** from an array to a dictionary.
4. Make necessary adjustments to the ***load_game*** functions for the dictionary conversion.
5. Adjust the ***encounter*** function to add a save feature when the player opts to run away.
6. Add feature to ***game_over*** to give user the option of playing the game again.

Development Standards:

1. Your Code must use a consistent variable naming convention.
2. All functions should be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a trademark of good code.

Commenting for Source Code Review (see the rubric):

1. Certain sections of your code will be graded. These gradable blocks of code can range from creating data structures, method algorithms, exception handling, and class construction.
2. The grading rubric contains a section called Source Code Review: in this section is a list of descriptions of all the graded programming concepts.
3. Each gradable item must have a comment placed at its beginning, and the comment must be prefixed with the comment flag. The flag helps the graders easily locate the code to increase the effectiveness of grading.
4. The flag will *must* use the naming convention **SC#** (NOTE: the # symbol will be replaced with sequential numbering such as **SC1**, **SC2**, **SC3**, etc.)
5. Unless stated otherwise, NO explanation in the comment with the flag is required, only the comment flag; however, any information placed in the comment could help the grader better understand and avoid any costly errors.
6. The comment flag needs to be placed near the block of code it represents.
7. **If a comment flag is not present, you will not receive credit.**
8. Here is an example: the Source Code Review has a gradable section of code for printing to the console (this is just a generic example, not on the actual rubric):
 - Rubric:
 - SC12: *print* method in the *main* class is printing the correct object ____10 pts
 - Code snippet:

```
#SC12 printing the car object
print(car)
```

Function Descriptions:

- A. Function **load_characters** needs to be adjusted so that the list of characters utilizes a dictionary instead of an array. The key for each character should be a number, starting with 1 for the first character.
- B. Function **save_game** has one parameter, the current character object. The function needs to do the following:
 - a. Set the current character's timestamp to the current time in the format YYYY-MM-DD HH:MM. Here is a sample of what the timestamp should look like:

```
2024-08-13 08:45
```

- b. Look through the list of characters to see if the current character appears in the list.
 - i. If the current character does appear in the list, display the following to the user:

```
Character save exists. Override (O), Make a new save (N), or Quit (Q)?
```

The gray box is where the user should enter either 'O', 'N', or 'Q' (this should not be case-sensitive). The user should only be allowed to enter one of these three options. Any other value should return the phrase "Invalid input. Please enter 'O', 'N', or 'Q'." and repeat the prompt.

1. If the user enters 'O', set the character at the existing key to the current character.
2. If the user enters 'N', create a new key and add the current character to the list.
3. If the user enters 'Q', function should end without adding current character to list.

- ii. If the current character is not found in the list, display:

```
No save found. Make a new save (N) or Quit (Q)?
```

The gray box is where the user should enter either 'N' or 'Q' (this should not be case-sensitive). The user should only be allowed to enter one of those two options. Any other value should return the phrase "Invalid input. Please enter 'N' or 'Q'." and repeat the prompt.

1. If the user enters 'N', create a new key and add the current character to the list.
 2. If the user enters 'Q', end the function without updating *GameSaves.txt* at all.
- c. Open *GameSaves.txt* and write each character from the list of characters into the file, matching the format of the characters already in the file. The function should then display:

```
Save Successful!
```

- C. Function ***load_game*** needs to be adjusted to account for the fact that the list of characters is now a dictionary instead of an array.
- a. The numbers displayed along with the character information in the load menu should be the keys from the dictionary so that the user's choice is the key of the character that needs to be loaded.
- D. Function ***encounter*** needs to be adjusted so that when the user enters a '2' to run away, the function *save_game* gets called.
- E. Function ***game_over*** needs to be adjusted so that at the end of the function, the user is asked if they would like to play again. The function should display:

```
Would you like to adventure again? (Y/N):
```

The gray box is where the user should enter either Y or N (it should not be case-sensitive). The user should only be allowed to enter one of those two options. Any other value should return the phrase "Invalid input. Please enter 'Y' or 'N'." and repeat the prompt.

- a. If the user enters 'Y', the function should restart the game by calling to *start_menu*
- b. If the user enters 'N', the function should display

```
See you on the next adventure!
```

and then the game should end.

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive / uploaded as a zipped folder. _____ 20 pts
Both required files are in one folder; the folder name is your contestant ID _____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB _____ 30 pts

If the program does not execute, then the remaining items in this section receive a score of zero.

Program prompts user to save when they opt to run away during an encounter. _____ 10 pts
If character already exists, the prompt will ask to override or create a new save. _____ 20 pts
Program should accept only 'O', 'N', or 'Q' (or 'o', 'n', or 'q') for this prompt. _____ 20 pts
If character doesn't exist yet, the prompt to save will ask to make a new save or quit. _____ 20 pts
Program should accept only 'N' or 'Q' (or 'n' or 'q') for this prompt. _____ 20 pts
When a save option is selected, program displays "Save Successful!" _____ 10 pts
When game ends, program prompts user to play again. _____ 30 pts
Program should accept only an 'Y' or 'N' (or 'y' or 'n') for this prompt. _____ 20 pts
When N/n is entered, program ends. _____ 20 pts
When a Y/y is entered, program restarts from the start menu. _____ 20 pts

Subtotal _____/250

Source Code Review

Contestant number is commented at the top of the Python file. _____ 10 pts

*NOTE: you **must** place the comment flag in front of the comment in your code **to get credit**. The comment flag will precede the explanation. For example, if the flag is SC1, your comment must read as "#SC1 ..." in front of the part of the code being reviewed. Code must work to get credit.*

SC1: Function **load_characters** successfully converts variable *characters* to a dictionary. _____ 30 pts
SC2: Keys for dictionary are integers, starting with 1. _____ 10 pts
SC3: Function **save_game** takes in a character object as a parameter. _____ 10 pts
SC4: Function **save_game** sets the current character timestamp in the format specified. _____ 20 pts
SC5: Function **save_game** utilizes the *datetime* import to set the timestamp. _____ 20 pts
SC6: Function **save_game** checks if current character exists in list of characters. _____ 20 pts
SC7: Function **save_game** uses conditionals to either replace existing character with new one _____ 20 pts
or create a new key and add current character to the list.
SC8: Function **save_game** writes list of characters to *GameSaves.txt*. _____ 30 pts
SC9: Function **load_game** adjusted to account for the change to a dictionary. The _____ 20 pts
numbers being displayed should be the keys of the dictionary. User's choice should
be one of the keys from the dictionary.
SC10: Function **encounter** calls on *save_game* when user opts to run away. _____ 20 pts
SC11: Function **game_over** uses a conditional to either call on *start_menu* or print a goodbye _____ 20 pts
statement depending on user response to wanting to play again.

Subtotal _____/230

Total Points _____/480

Completed Source Code

```
import random
import datetime

def save_game(character):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")
    character.timestamp = timestamp # Update character's timestamp

    # Generate a new unique key for the character save
    new_key = len(characters) + 1

    found = False

    for key, char in characters.items():
        if char.name == character.name:
            found = True
            while True:
                choice = input("Character save exists. Override (O), Make a new save (N), or Quit (Q)? ").upper()
                if choice == 'O' or choice == 'N' or choice == 'Q':
                    break
                else:
                    print("Invalid input. Please enter 'O', 'N', or 'Q'.")
            if choice == 'O':
                new_key = key # Override the existing save with the same key
                break
            if choice == 'Q':
                return
    if found == False:
        while True:
            choice = input("No save found. Make a new save (N) or Quit (Q)? ").upper()
            if choice == 'Q' or choice == 'N':
                break
            else:
                print("Invalid input. Please enter 'N' or 'Q'.")
        if choice == 'Q':
            return

    characters[new_key] = character # Add or update character save

    # Update the GameSaves.txt file
    with open('NatGameSaves.txt', 'w') as file:
        for char in characters.values():

            file.write(f"{char.name},{char.weapon},{char.level},{char.strength},{char.health},{char.gold},{char.timestamp}\n")
    print("Save Successful!")

class Character:
    def __init__(self, name, weapon, level=1, strength=random.randint(12, 17), health=random.randint(7, 10) * 10, gold=0, timestamp=""):
        self.name = name
        self.weapon = weapon
        self.level = level
```

```

self.strength = strength
self.health = health
self.gold = gold
self.timestamp = timestamp

```

```

def load_characters(filename):
    characters = {}
    number = 0
    with open(filename, 'r') as file:
        for line in file:
            number += 1
            name, weapon, level, strength, health, gold, timestamp = line.strip().split(',')
            character = Character(name, weapon, int(level), int(strength), int(health), int(gold), timestamp)
            characters[number] = character
    return characters

```

***OLD FUNCTION FROM STATE TEST TO COMPARE WITH ***

```

def load_characters(filename):
    characters = []
    with open(filename, 'r') as file:
        for line in file:
            name, weapon, level, strength, health, gold, timestamp = line.strip().split(',')
            characters.append(Character(name, weapon, int(level), int(strength), int(health), int(gold), timestamp))
    return characters

```

```

def start_menu():
    while True:
        print("BPA Dungeon Adventure")
        print("-----")
        print("(N) New Game\n(L) Load Game")
        choice = input("Enter your choice (N/L): ").upper() # Converts the input to uppercase to handle lowercase inputs
        print()
        if choice == 'L':
            characters = load_characters('NatGameSaves.txt')
            load_game(characters)
            break # Break out of the loop if a valid option is chosen
        elif choice == 'N':
            start_new_game()
            break # Break out of the loop if a valid option is chosen
        else:
            print("Invalid choice!\n")

```

```

def start_new_game():
    print("Welcome to the dungeon!\nTreasure awaits for those brave enough to venture inside...")
    name = input("Enter your character's name: ")
    weapon = input("Choose your weapon: ")
    character = Character(name, weapon)
    start_game(character)

```

```

def load_game(characters):
    header = "There are " + str(len(characters)) + " available saves:"

```

```

print()
print(header)
print("-" * len(header))

for key, char in characters.items():
    line = f"{key:>3}. {char.name}\tLevel {char.level}\t{char.timestamp:>20}"
    print(line)

while True: # Loop to allow re-selection if user declines
    try:
        choice = int(input("\nChoose a save to load: "))
        if 0 < choice <= len(characters):
            selected_char = Character(characters[choice].name, characters[choice].weapon, characters[choice].level,
                                      characters[choice].strength, characters[choice].health, characters[choice].gold,
                                      characters[choice].timestamp)
            display_character(selected_char) # Show chosen character details
            confirm = input("Load this character? (Y/N): ").upper()
            if confirm == 'Y':
                # Move on with that character
                start_game(selected_char)
                break # Exit loop if confirmed
            elif confirm == 'N':
                continue # Re-display list of saves for a new choice
            else:
                print("Invalid input. Please enter 'Y' or 'N'.")
        else:
            print("Invalid selection.\nPlease choose a number from the list.")
    except ValueError:
        print("Invalid input. Please enter a number.")

def display_character(char):
    print(f"\nName: {char.name}\t\tLevel: {char.level}")
    print(f"Gold: {char.gold:02d}\t\tWeapon: {char.weapon}")
    print(f"Health: {char.health}\t\tStrength: {char.strength}\n\n")

def found_loot(char):
    loot = random.choice('ggggggghhh')
    if loot == 'g':
        new_gold = random.randint(25, 150)
        char.gold += new_gold
        return f"{new_gold} gold"
    else:
        bonus = random.randint(1, 3) * 10
        char.health += bonus
        return f"a health potion. You restored {bonus} health"

def encounter(char):
    print("A monster is attacking you!")
    while True:
        try:
            attack = int(input(f"Enter:\t'1' to use your {char.weapon},\n\t'2' to run away\nChoice: "))
            if attack < 1 or attack > 2:
                print("Invalid choice!")

```

```

        else:
            break
    except ValueError:
        print("Invalid Choice!")
if attack == 1:
    monster = random.randint(10, 20)
    result = char.strength - monster
    if result >= 0:
        reward = found_loot(char)
        input(f"\nYou defeated the monster and found {reward}!\nPress Enter to continue")
    else:
        char.health -= abs(result) * 10
        print(f"\nThat was rough! You lost {abs(result) * 10} health.")
        if char.health <= 0:
            char.health = 0
        else:
            input("Luckily you managed to get past the monster!\nPress Enter to continue")
    else:
        save_game(char)
return attack

def game_over(char):
    if char.health > 0 and char.level == 4:
        treasure = random.randint(500, 5000)
        char.gold += treasure
        print(f"\nYou made it to the treasure! You found {treasure} gold!")
    elif char.health > 0 and char.level < 4:
        print("\nYou didn't find the treasure, but you survived to fight again another day...")
    else:
        print("\nYou fought as best you could, but didn't make it.\nThe treasure waits for the next adventurer...")
    display_character(char)
    while True:
        repeat = input("Would you like to adventure again? (Y/N): ").upper()
        if repeat != 'Y' and repeat != 'N':
            print("Invalid input. Please enter 'Y' or 'N'.")
        elif repeat == 'Y':
            start_menu()
            break
        else:
            print("See you on the next adventure!")
            break

def start_game(character):
    display_character(character)
    # rewrite the for loop to pick up where the character left off
    for i in range(character.level, 4):
        result = encounter(character)
        if character.health <= 0 or result == 2:
            break
        else:
            character.level += 1
            display_character(character)

```

```
game_over(character)
```

```
# Main code
```

```
characters = load_characters("NatGameSaves.txt")
```

```
start_menu()
```